

Black Cherry Trees: Girth vs. Volume

R fits the model · Python re-renders it · one shared dataset

The data and the fit

R's built-in trees dataset records girth, height, and usable timber volume for 31 black cherry trees.

```
data(trees)
fit <- lm(Volume ~ Girth, data = trees)
slope <- coef(fit)[2]
r2 <- summary(fit)$r.squared
pred <- fitted(fit) # plain numeric vector, for the Python bridge below
```

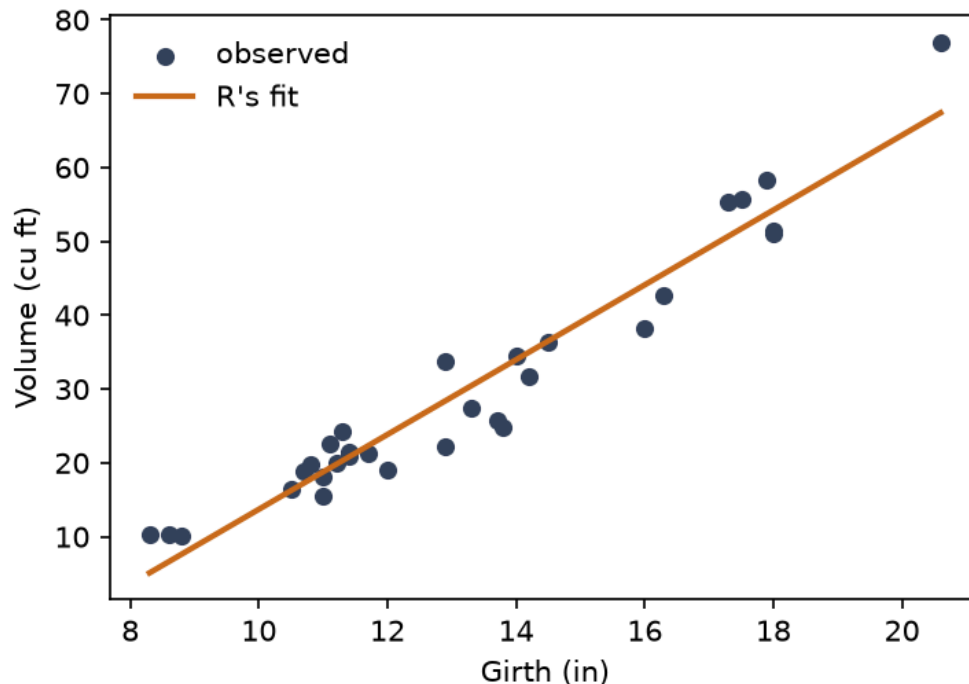
A simple linear fit puts the slope at **5.0659 cubic ft of volume per inch of girth** ($R^2 = 0.9353$) — girth alone is a strong stand-in for a tree's timber yield.

The same fit, from Python

```
import numpy as np
import matplotlib
matplotlib.use("Agg") # headless-safe backend; plt.show() just discards
import matplotlib.pyplot as plt

# r.trees and r.pred reach directly into the R chunk above -- no CSV
# round-trip, no re-fitting from scratch.
girth = np.array(r.trees["Girth"])
volume = np.array(r.trees["Volume"])
pred = np.array(r.pred)

order = np.argsort(girth)
plt.figure(figsize=(5, 3.6))
plt.scatter(girth, volume, color="#33415c", s=28, label="observed")
plt.plot(girth[order], pred[order], color="#c96a1a", lw=2, label="R's fit")
plt.xlabel("Girth (in)")
plt.ylabel("Volume (cu ft)")
plt.legend(frameon=False)
plt.tight_layout()
plt.show()
```



```
residual_spread = float(np.std(volume - pred))
```

Python isn't re-fitting anything here — `r.pred` is the fitted-values vector R just computed, read straight across the bridge and re-drawn in matplotlib's style. The residual spread computed on the Python side, 4.1125 cubic ft, is what R reports next.

Back in R

```
py_resid <- reticulate::py$residual_spread
```

Typical prediction error is about $\bar{e} = 4.1125$ cubic ft — small enough, relative to a volume range of 66.8 cubic ft, that girth alone is a serviceable field estimate for timber volume.

A value genuinely inside an equation

The trick above (a symbol in math, its value right next to it in prose) only works at the **edge** of a math expression. A standard-error formula needs its values in the middle — under a square root, inside a fraction — which has no edge to sit at. For that, tveave's `typst_vars()` exposes a whole dictionary of values as real Typst source, referenced with `val("key")` anywhere in the expression, including deep inside one:

```
mse <- summary(fit)$sigma^2
sxx <- sum((trees$Girth - mean(trees$Girth))^2)
t_crit <- qt(0.975, df = fit$df.residual)
```

$$\beta_1 \pm t^* \cdot \sqrt{\frac{0}{0}}$$

which gives a 95% CI of approximately (4.56, 5.572) cubic ft per inch of girth. `val()` (imported from `tweave`, alongside `vals` itself) is a shorthand for `vals.at(key, default: 0)`, so there's no need to repeat the default at every use site – and no placeholder to add at the top of the document either, since `vals` starts out empty the moment you import `tweave`, before any chunk has run.